

Patrones de Modelado de Procesos

Ingeniería en Sistemas de Información

UTN – F. R. Rosario

Bibliografía Patrones de Modelado de Procesos

- Patrones de Modelado de Procesos - Bizagi
 - https://www.bizagi.com/docs/Workflow_Patterns_using_BizAgi_Process_Modeler_Esp.pdf
- Workflow Patterns
 - <http://www.workflowpatterns.com/>

Características de un patrón

- Describe un problema y su solución
- No se enfoca en un dominio o situación específica. Generaliza para muchas situaciones similares
- Soluciones reutilizables a problemas comunes
- Resume la experiencia de expertos y de las mejores prácticas
- Ayudan a aprender de los éxitos de otros, en lugar de los errores propios
- Lenguaje común

Patrones de Modelado de Procesos

- Patrones de Flujo de Control
 - Capturan la forma en que los flujo de control son representados y usados en procesos
- Patrones de Datos
 - Capturan la forma en que los datos son representados y usados en procesos
- Patrones de Recursos
 - Capturan la forma en que los recursos son representados y usados en procesos

Patrones de Flujo de Control

- Los Patrones de Flujo de Control proveen un enfoque práctico para entender y analizar los flujo de control de procesos
- Las categorías de los Patrones de Flujo de Control son:
 - Patrones básicos de flujo de control
 - Patrones avanzados para ramificación y sincronización
 - Patrones de múltiples instancias
 - Patrones basados en estados
 - Patrones de cancelación
 - Patrones de iteraciones
 - Patrones de terminación

Patrones básicos de flujo de control

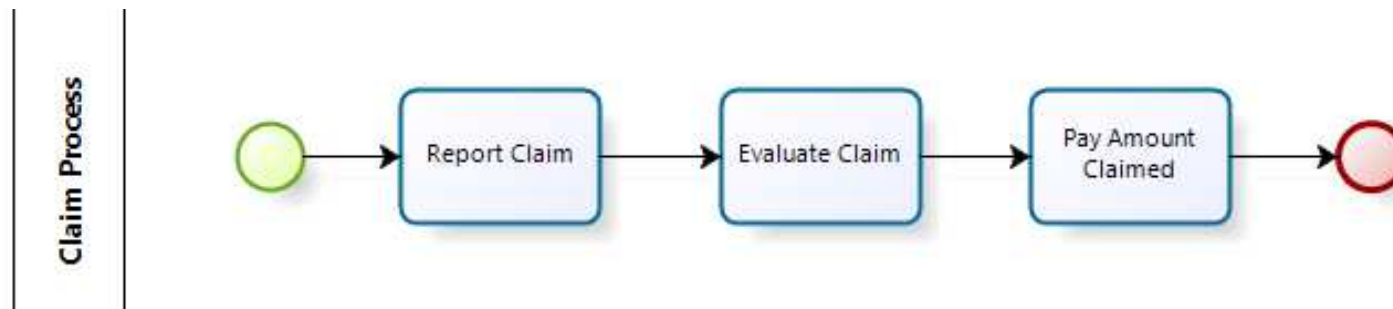
- Patrón 1 – Secuencia

- Descripción:

- Este patrón es utilizado para modelar dependencia entre tareas, es decir, una tarea no puede empezar hasta que otra no haya terminado

- Implementación:

- Para modelar este patrón es necesario conectar las actividades (en el orden de ejecución definido) utilizando conectores de flujo de secuencia

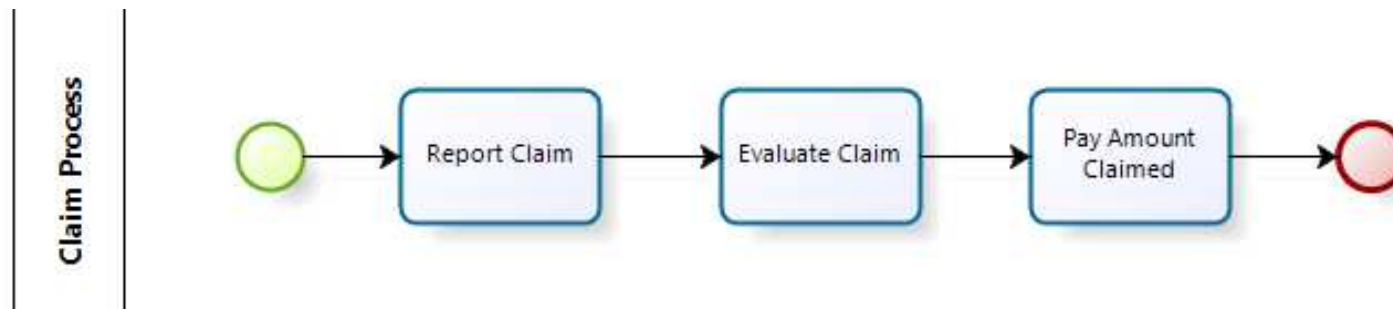


Patrones básicos de flujo de control

- Patrón 1 – Secuencia

- Ejemplo:

- Cuando una compañía de seguros recibe un reclamo en alguna póliza, es necesario realizar varias actividades con un orden definido. Primero, el cliente debe reportar el reclamo, luego, éste debe ser evaluado para poderme autorizar el pago y finalmente se desembolsa la cantidad reclamada.
 - No tendría sentido que una reclamación sea evaluada antes de ser reportada, o pagar el dinero de la reclamación antes de evaluarla.



Patrones básicos de flujo de control

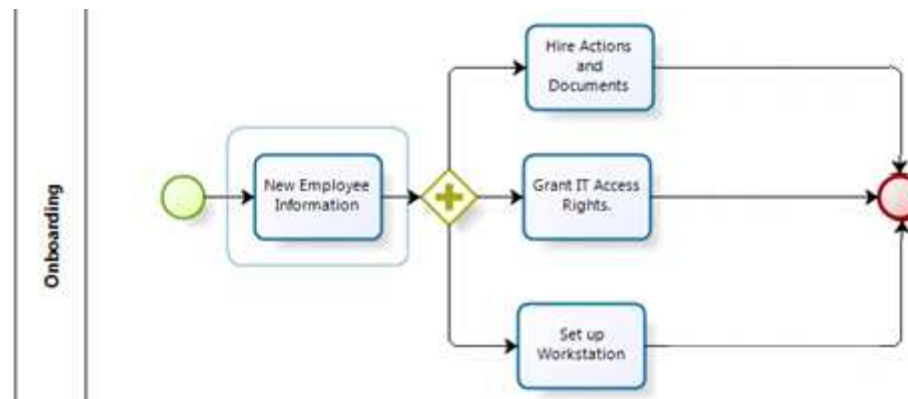
- Patrón 2 – División Paralela

- Descripción:

- Una división paralela es un punto del proceso donde un camino es dividido en dos o más ramas paralelas las cuales son ejecutadas al mismo tiempo

- Implementación:

- Para implementar la División Paralela, es necesario utilizar la **Compuerta Paralela**. Esta compuerta activa caminos alternativos sin verificar condiciones.

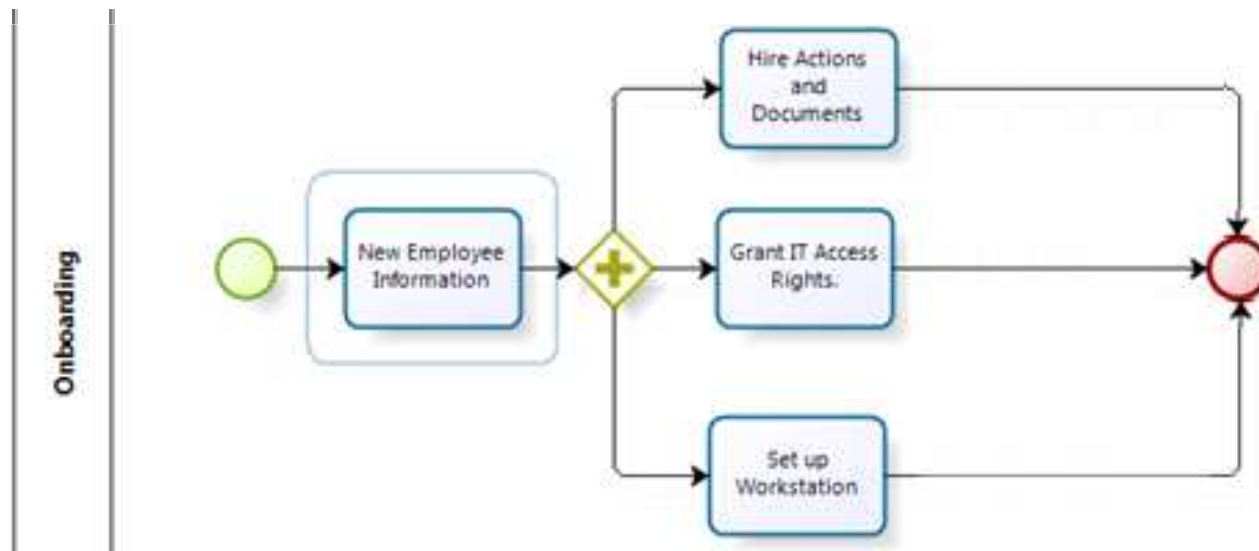


Patrones básicos de flujo de control

- Patrón 2 – División Paralela

- Ejemplo:

- Cuando un nuevo empleado llega a la compañía es necesario realizar varias actividades, por ejemplo, darle acceso a la información de su cargo, firmar algunos documentos legales y alistar su puesto de trabajo.



Patrones básicos de flujo de control

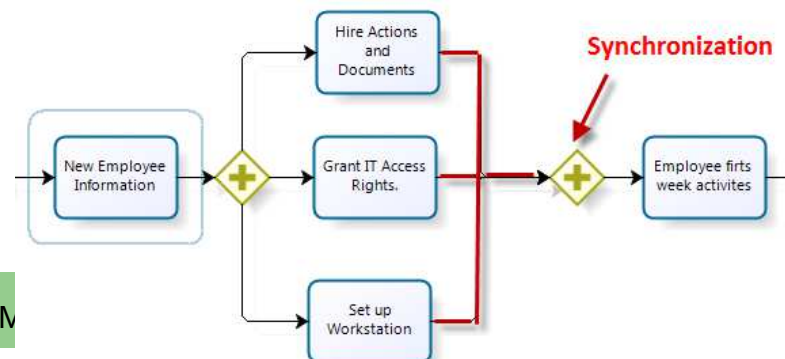
- Patrón 3 – Sincronización

- Descripción:

- La sincronización es un punto en el proceso donde dos o más ramas del proceso se unen en una sola. Se llama sincronización porque se espera a que todas las ramas entrantes se completen antes de continuar con la siguiente actividad

- Implementación:

- Para modelar este patrón se puede utilizar una Compuerta Paralela como elemento convergente para lograr la unión de todas las ramas. El patrón de sincronización también puede ser modelado utilizando compuertas inclusivas o exclusivas dependiendo de los requerimientos del negocio.

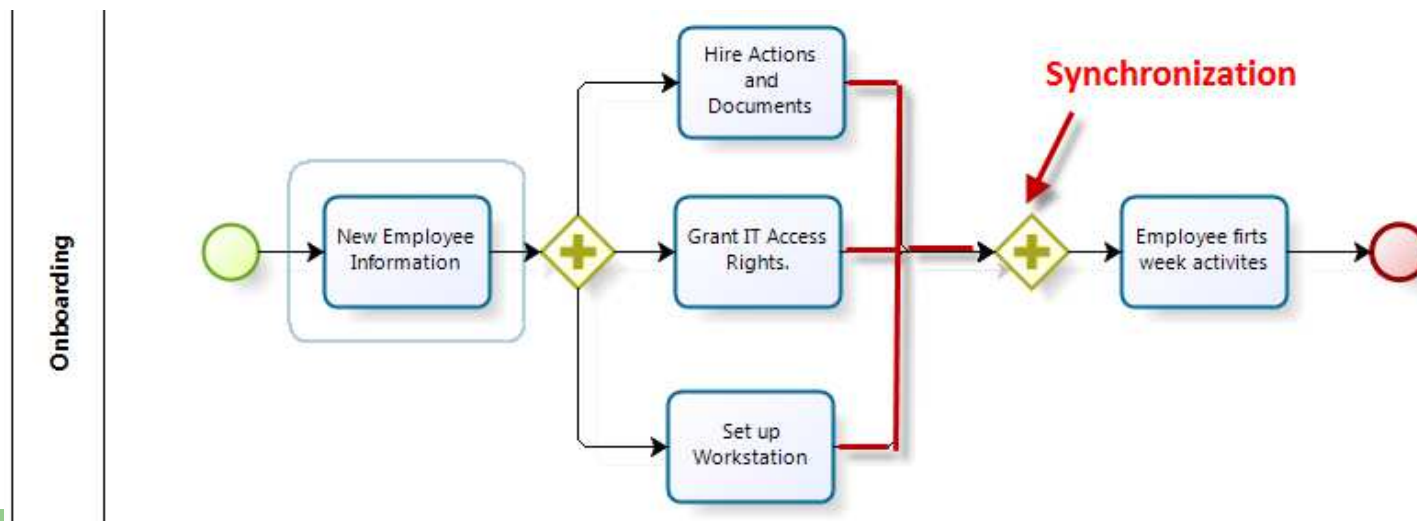


Patrones básicos de flujo de control

- Patrón 3 – Sincronización

- Ejemplo:

- Cuando un nuevo empleado llega a la compañía es necesario realizar varias actividades, por ejemplo, darle acceso a la información de su cargo, firmar algunos documentos legales y alistar su puesto de trabajo.
 - El empleado no puede empezar a trabajar hasta que todas las actividades se hayan completado.



Patrones básicos de flujo de control

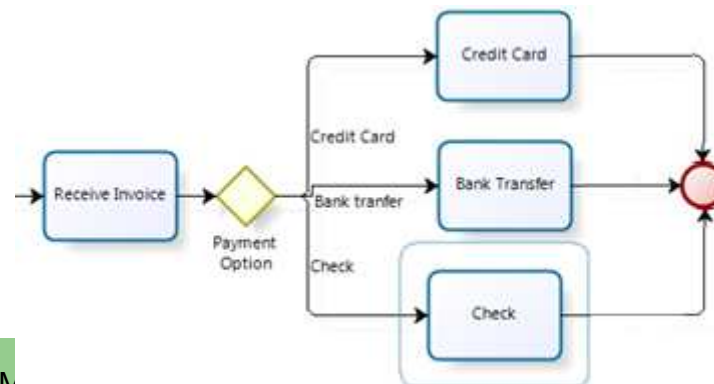
- Patrón 4 – Decisión Exclusiva

- Descripción:

- La decisión exclusiva representa un punto en el proceso donde se debe escoger un solo camino de varios disponibles dependiendo de una decisión o de datos del proceso

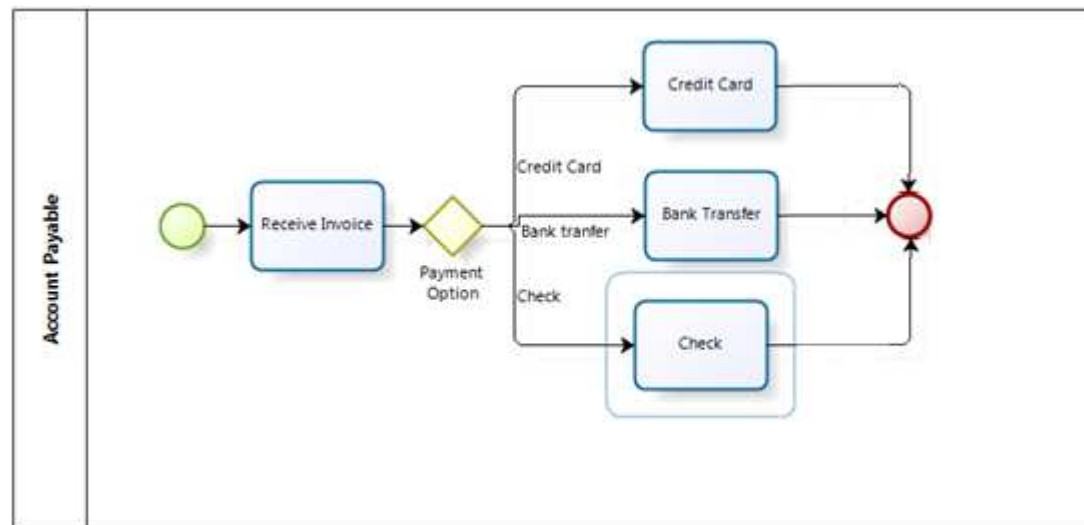
- Implementación:

- El patrón de decisión exclusiva puede ser modelado a través de una **Compuerta Exclusiva**. Para este caso la compuerta tiene tres secuencias de flujo pero el proceso solo debe continuar por uno de ellos de acuerdo al cumplimiento de la condición definida.



Patrones básicos de flujo de control

- Patrón 4 – Decisión Exclusiva
 - Ejemplo:
 - El Departamento de Contabilidad recibe y paga las facturas de la compañía. El pago de una factura se puede realizar a través de transferencia bancaria, cheque o tarjeta de crédito. Solo es posible realizar el pago de la factura con un medio de pago.



Patrones básicos de flujo de control

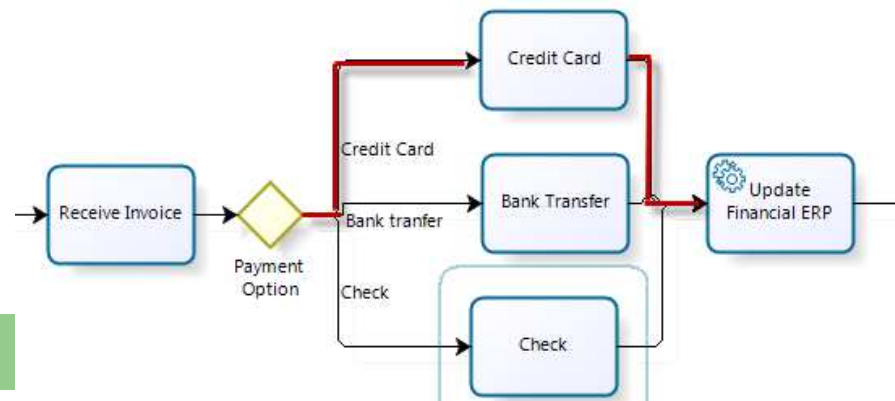
- Patrón 5 – Unión Simple

- Descripción:

- La unión simple es un punto del proceso donde dos o más caminos alternativos convergen en uno solo. Es un supuesto de este patrón que los caminos alternativos no son ejecutados en paralelo

- Implementación:

- Para realizar la implementación de este patrón se utilizan conectores de flujo de secuencia para conectar las actividades de las ramas entrantes con la siguiente actividad

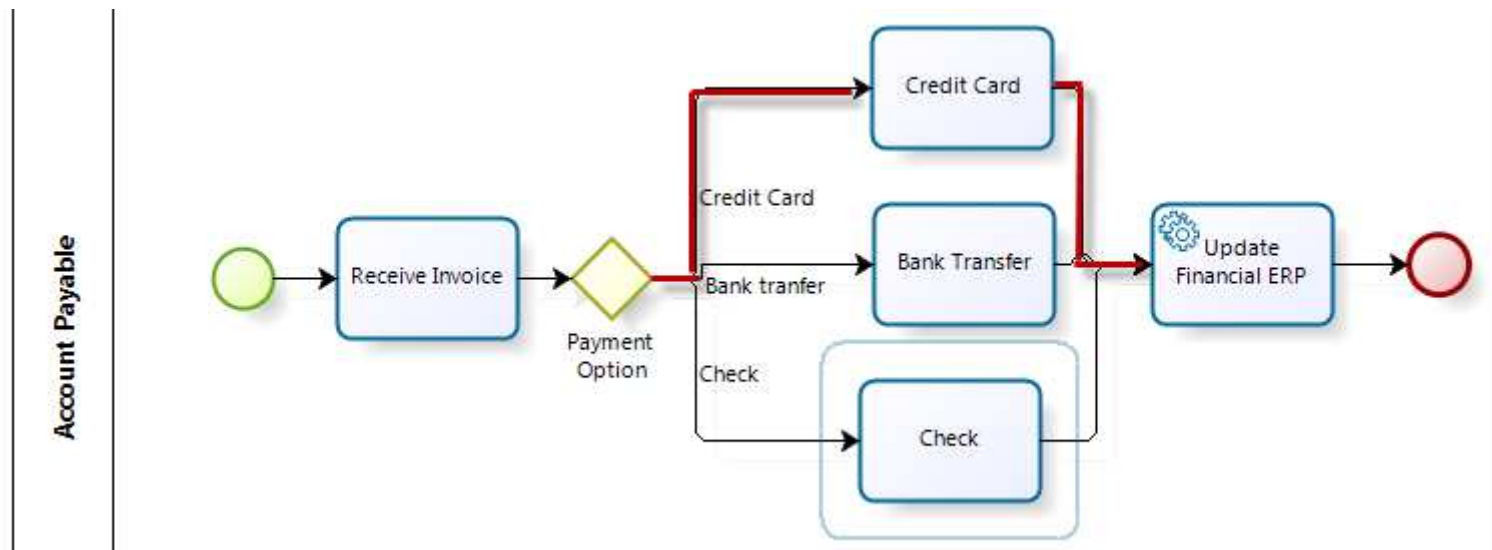


Patrones básicos de flujo de control

- Patrón 5 – Unión Simple

- Ejemplo:

- Tomemos el ejemplo que utilizamos en el patrón 4. Suponga que luego de realizar el pago con alguna de las opciones disponibles el ERP financiero es actualizado. Note en el diagrama 4 que la actividad Actualizar ERP Financiero solo será ejecutada una vez, esto debido a que solo una de las ramas entrantes es activada.



Patrones avanzados para ramificación y sincronización

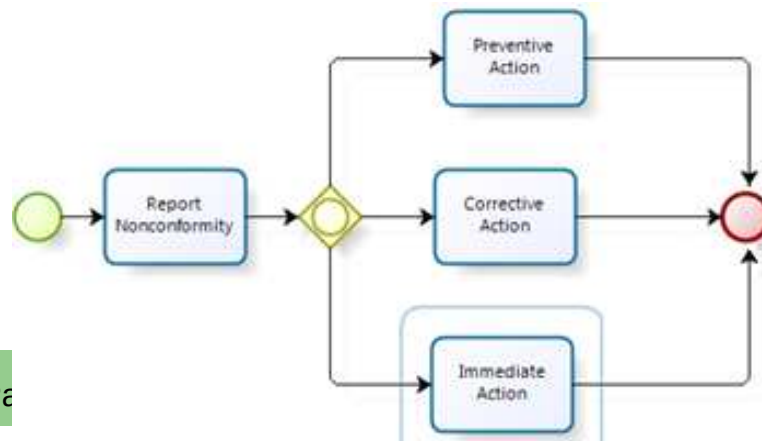
- Patrón 6 – Elección Múltiple

- Descripción:

- El patrón de elección múltiple es utilizado para modelar puntos en el flujo de proceso donde varios caminos son escogidos dependiendo de datos del proceso y/o puntos de decisión

- Implementación:

- Para implementar el patrón de elección múltiple se utiliza una **Compuerta Inclusiva**. Ésta compuerta es utilizada para activar uno o más caminos dependiendo de datos del proceso

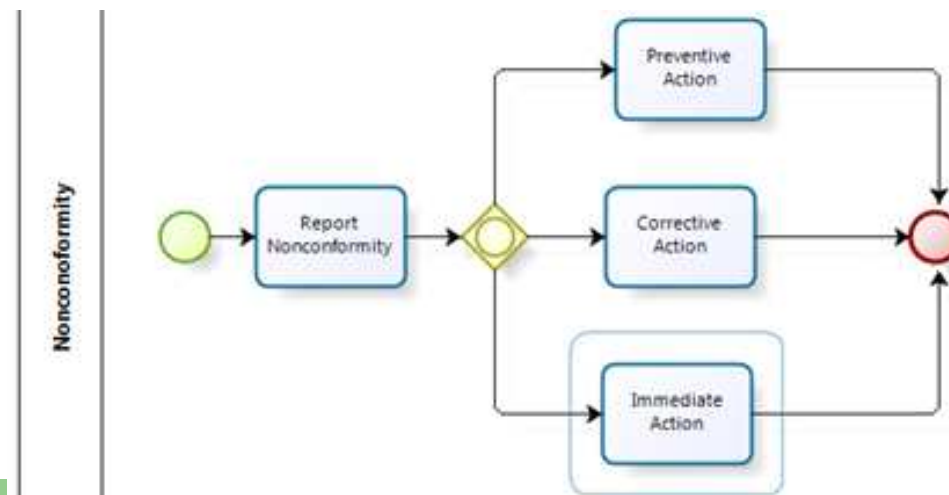


Patrones avanzados para ramificación y sincronización

- Patrón 6 – Elección Múltiple

- Ejemplo:

- Durante un proceso de auditoria es muy común encontrar no conformidades dentro del proceso auditado. Las no conformidades deben ser evaluadas y corregidas por el dueño del proceso. La corrección de una no conformidad puede ser realizada de diferentes formas, por ejemplo, con una acción correctiva, una acción preventiva, una acción inmediata o una combinación entre las opciones anteriores.



Patrones avanzados para ramificación y sincronización

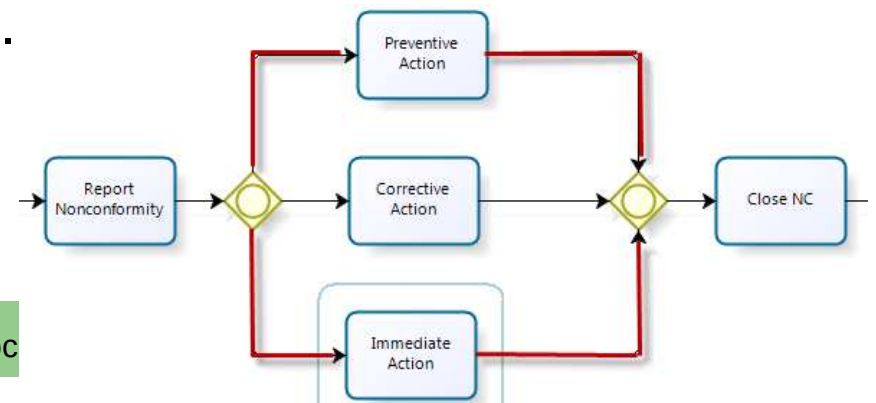
- Patrón 7 – Unión Sincronizada

- Descripción:

- Es un punto en el proceso donde múltiples caminos que fueron activados antes en el proceso convergen en una sola rama, el proceso continua hasta que todas las actividades de las ramas entrantes hayan terminado

- Implementación:

- Para implementar el patrón se deben utilizar **dos compuertas inclusivas**, una como elemento divergente (activar algunas de las ramas salientes) y otro como elemento de sincronización o convergente (para esperar que todas las ramas activadas lleguen a un determinado punto para continuar con el proceso).

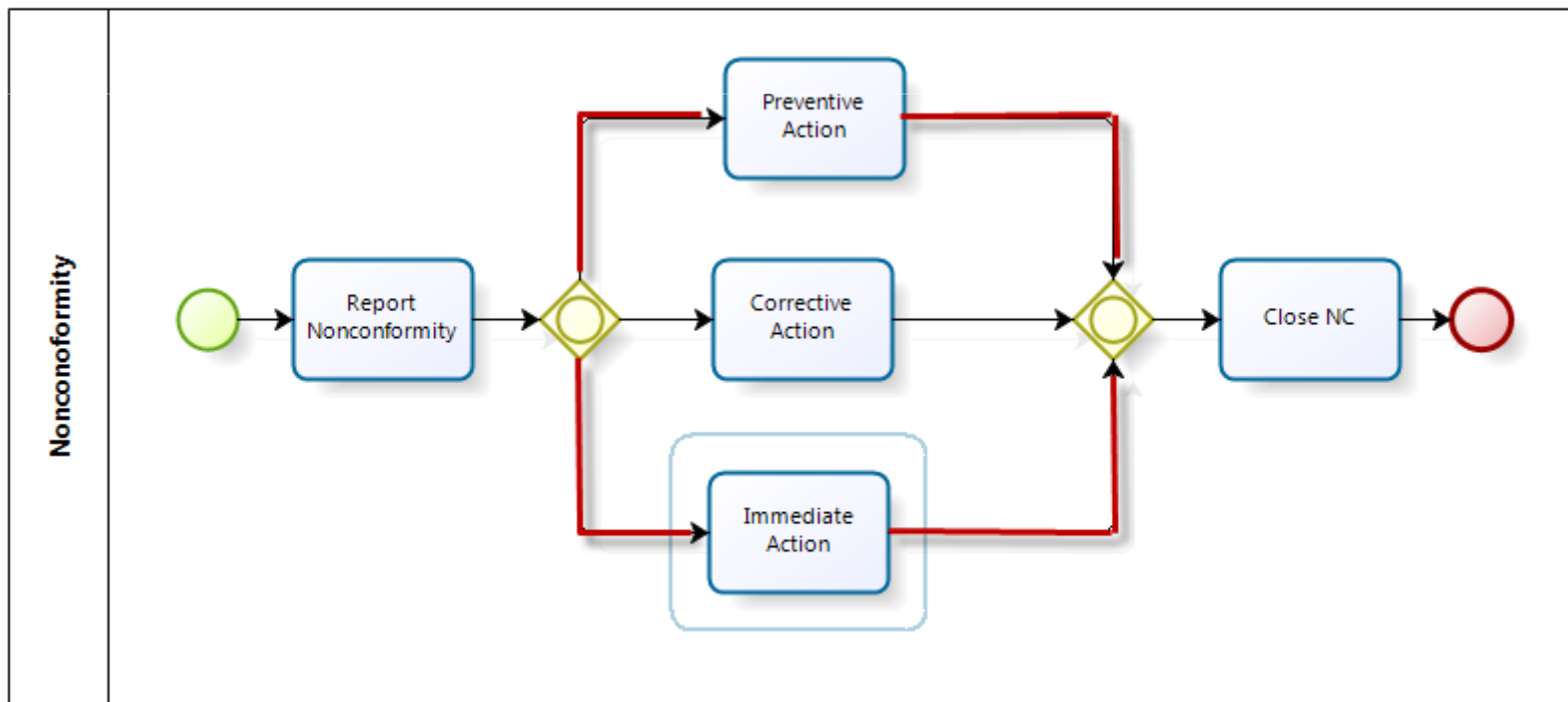


Patrones avanzados para ramificación y sincronización

- Patrón 7 – Unión Sincronizada

- Ejemplo:

- Continuando con el ejemplo utilizado en Patrón 4, suponga ahora que la no conformidad no puede ser cerrada hasta que las actividades que fueron seleccionadas hayan terminado



Patrones avanzados para ramificación y sincronización

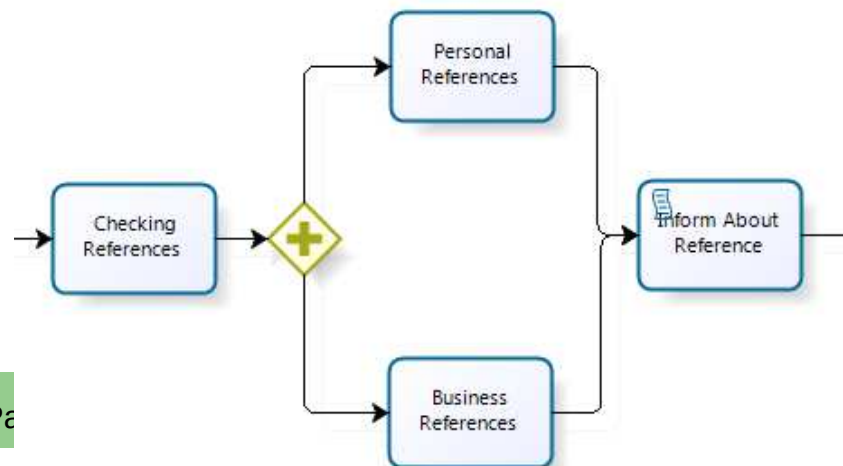
- Patrón 8 – Unión Múltiple

- Descripción:

- El patrón de unión múltiple es usado para converger dos o más ramas en un solo camino. Cada activación de una de las ramas entrantes resulta en la activación de la siguiente actividad en el proceso

- Implementación:

- El patrón utiliza una compuerta paralela para habilitar los dos caminos. Cada vez que la actividad de uno de los caminos es terminada, la siguiente actividad es ejecutada.

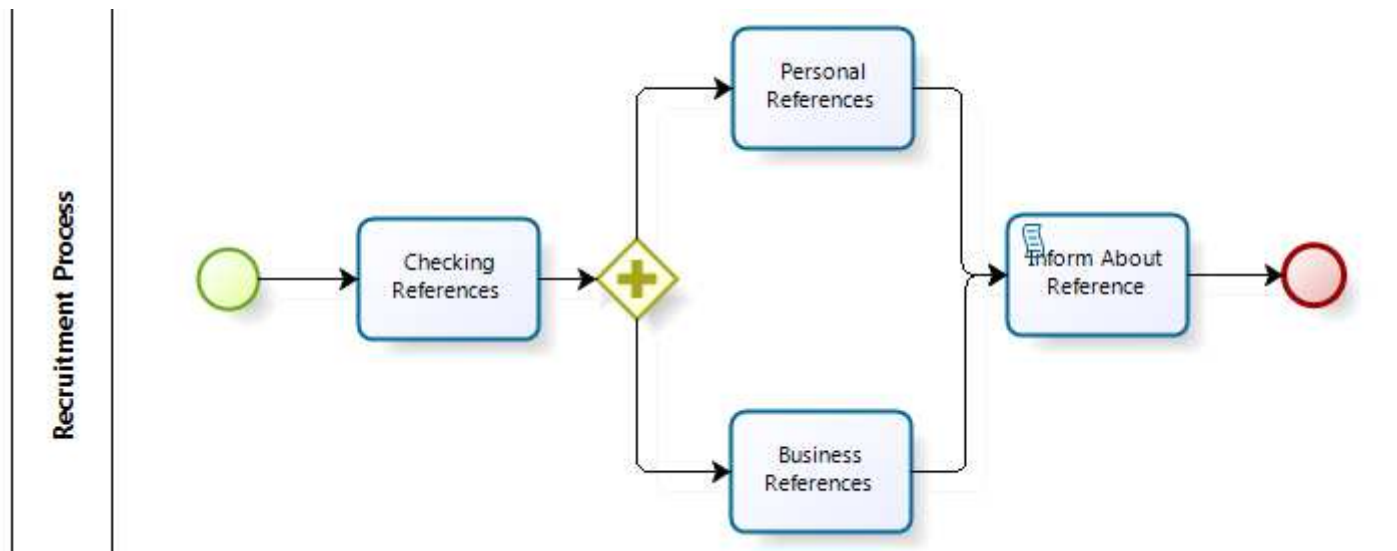


Patrones avanzados para ramificación y sincronización

- Patrón 8 – Unión Múltiple

- Ejemplo:

- Durante el Proceso de Selección de personal es necesario verificar las referencias del nuevo empleado. Es muy importante verificar las referencias personales y laborales suministradas. Cada vez que una referencia es verificada, el Jefe de Recursos Humanos debe ser notificado.



Patrones avanzados para ramificación y sincronización

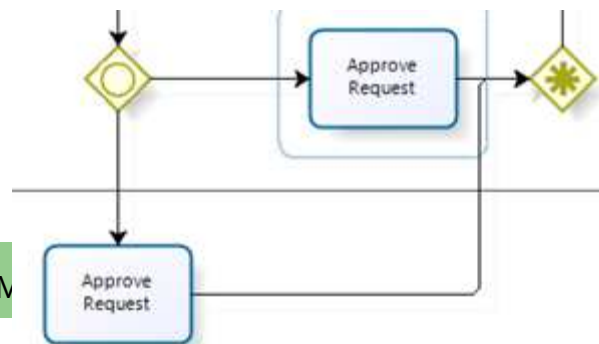
- Patrón 9 – Discriminador Estructurado

- Descripción:

- Este patrón describe un punto en el proceso donde se espera a que una de las ramas entrantes haya sido completada antes de continuar con la siguiente actividad, las otras ramas son omitidas después de ser completadas. Una vez todas las ramas entrantes han sido completadas, el discriminador se reinicia

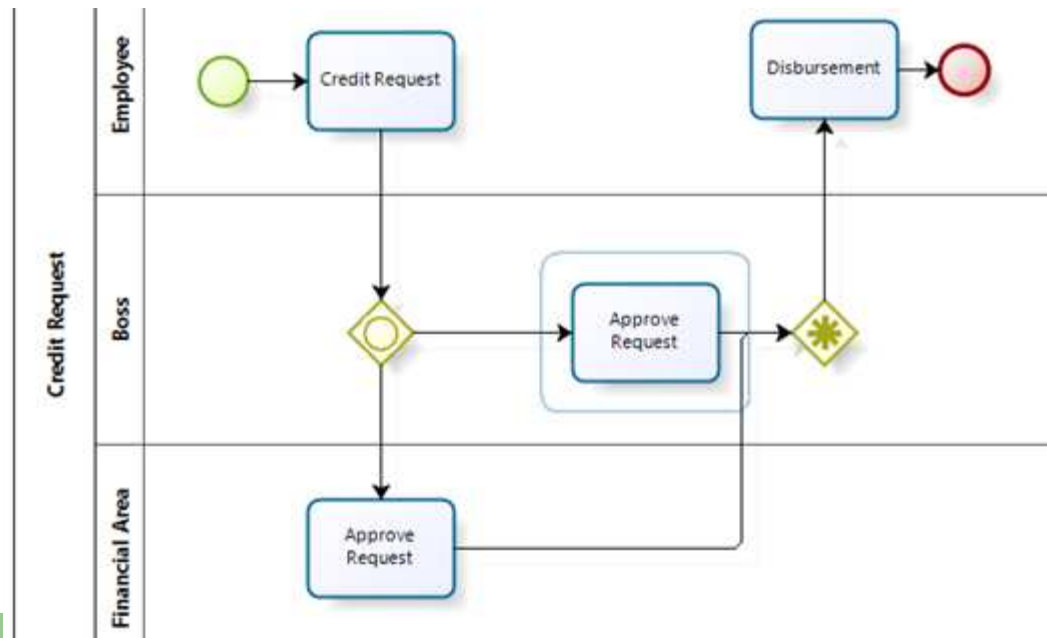
- Implementación:

- Este patrón puede utilizar compuertas Paralelas o Inclusivas para dividir los caminos. Para modelar el discriminador es necesario utilizar una **Compuerta Compleja** la cual espera hasta que una de las dos ramas entrantes sea completada



Patrones avanzados para ramificación y sincronización

- Patrón 9 – Discriminador Estructurado
 - Ejemplo:
 - Un empleado solicita un préstamo a su compañía. Para otorgar el crédito es necesario que la solicitud sea aprobada por su jefe o por el jefe del área financiera. Cuando alguno de los dos de su aprobación, el dinero puede ser desembolsado al empleado



Patrones de múltiples instancias

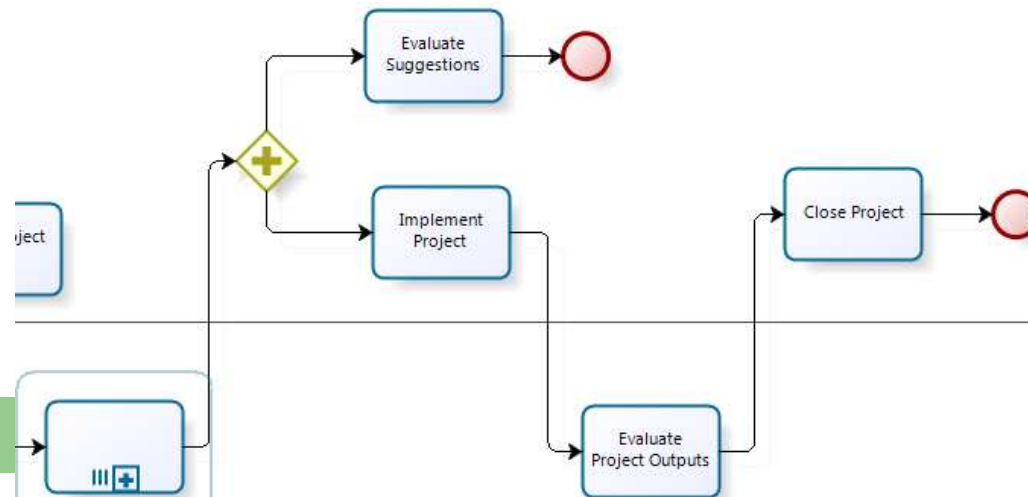
- Patrón 12 – Múltiples instancias sin Sincronización

- Descripción:

- El patrón es utilizado para modelar actividades que han sido instanciadas muchas veces en el proceso y no necesitan ser sincronizadas para que el flujo continúe.

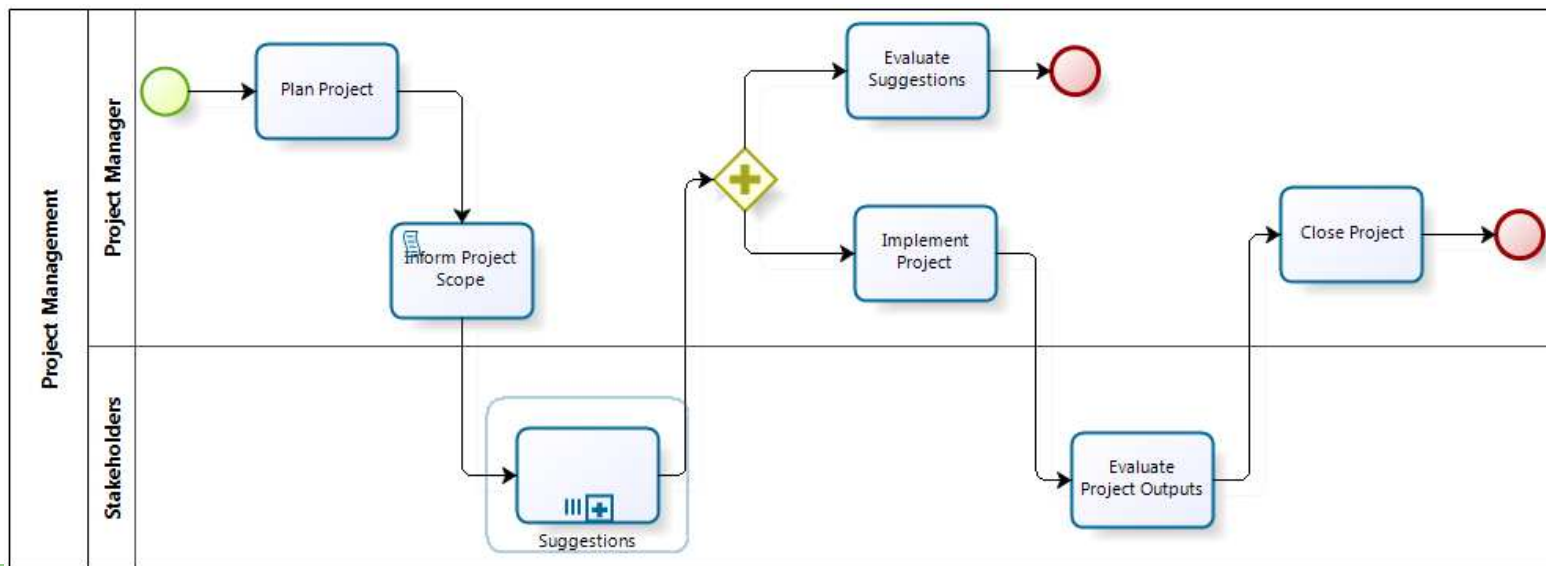
- Implementación:

- Para modelar este proceso es necesario utilizar una actividad (sub proceso o tarea) que crea múltiples instancias. El proceso continúa con su flujo normal cuando la actividad (sub proceso o tarea) ha sido activada.



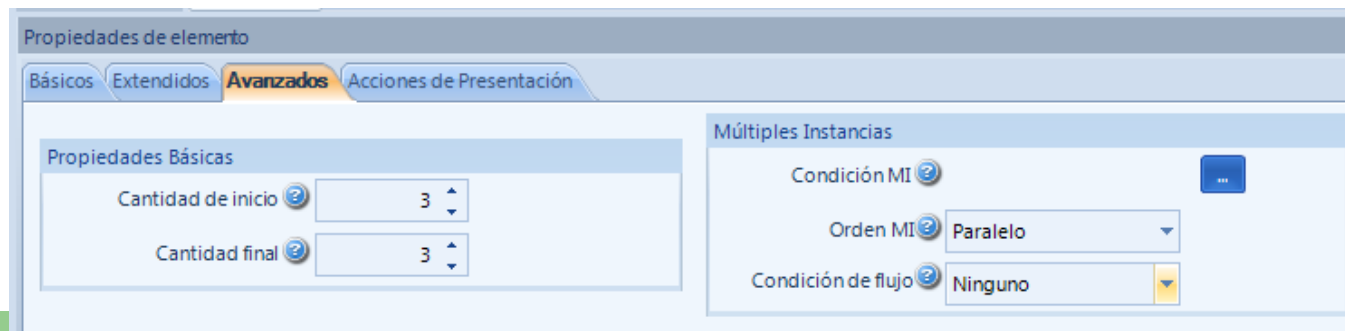
Patrones de múltiples instancias

- Patrón 12 – Múltiples instancias sin Sincronización
 - Ejemplo:
 - Una compañía está planeando implementar un nuevo proyecto. Las partes interesadas (Stakeholders) son informadas sobre el alcance del proyecto y pueden enviar sus comentarios o sugerencias sobre él si así lo desean. El proceso continúa aunque no se reciba ninguna sugerencia



Patrones de múltiples instancias

- Patrón 12 – Múltiples instancias sin Sincronización
 - Para configurar el comportamiento del subproceso sugerencias para la condición que se desea, es necesario definir las siguientes propiedades:
 - **Cantidad de Inicio = 3.** Esto significa que el proceso debe ser instanciado 3 veces. Suponga que existen 3 Stakeholders.
 - **Cantidad Final= 3.** El sub proceso se considera como terminado cuando se hayan completado todas (en este caso) las 3 instancias.
 - **Condición MI= Ninguna.** Esto quiere decir que no hay condición para el número de instancias para el sub proceso.
 - **Orden MI = Paralelo.** Las instancias del sub proceso serán ejecutadas en paralelo.
 - **Condición de Flujo = Ninguno.** El proceso continúa una vez se lanzan las instancias del sub proceso.



Propiedades de elemento

Básicos Extendidos **Avanzados** Acciones de Presentación

Propiedades Básicas

Cantidad de inicio 3

Cantidad final 3

Múltiples Instancias

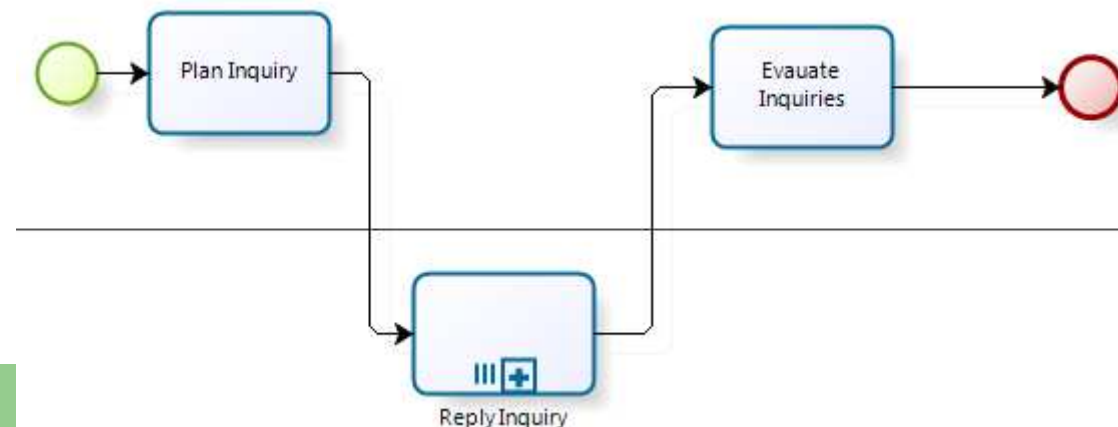
Condición MI

Orden MI Paralelo

Condición de flujo Ninguno

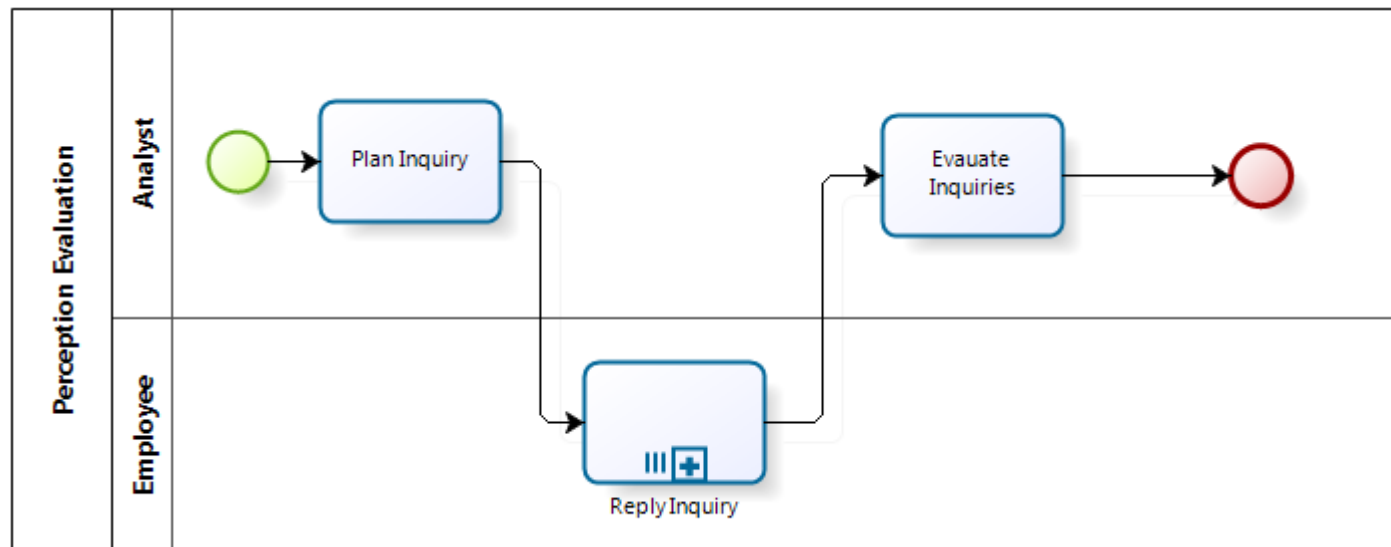
Patrones de múltiples instancias

- Patrón 13 – Múltiples instancias con conocimiento a priori del número de ejecuciones en la etapa de diseño
 - Descripción:
 - El patrón permite instanciar una actividad varias veces en el proceso. El número de instancias es conocido y se ejecutan al mismo tiempo. Es necesario terminar todas las actividades antes de continuar con el flujo del proceso
 - Implementación:
 - Para implementar este patrón es necesario utilizar una tarea o un sub proceso para crear las instancias de la actividad



Patrones de múltiples instancias

- Patrón 13 – Múltiples instancias con conocimiento a priori del número de ejecuciones en la etapa de diseño
 - Ejemplo:
 - Una compañía decide evaluar la percepción del ambiente de trabajo entre los empleados. Una encuesta es enviada a 100 empleados. Cuando todos los empleados hayan enviado sus resultados, el analista puede evaluar la información.



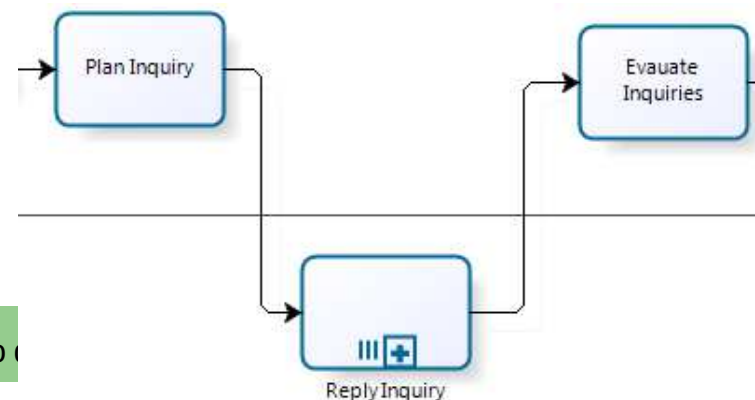
Patrones de múltiples instancias

- Patrón 13 – Múltiples instancias con conocimiento a priori del número de ejecuciones en la etapa de diseño
 - Para configurar el comportamiento del subproceso sugerencias para la condición que se desea, es necesario definir las siguientes propiedades:
 - **Cantidad de Inicio = 100.** Esto significa que el proceso debe ser instanciado 100 veces.
 - **Cantidad Final= 100.** El sub proceso se considera como terminado cuando se hayan completado todas (en este caso) las 100 instancias.
 - **Condición MI= Ninguna.** Esto quiere decir que no hay condición para el número de instancias para el sub proceso.
 - **Orden MI = Paralelo.** Las instancias del sub proceso serán ejecutadas en paralelo.
 - **Condición de Flujo = Todo.** El proceso continúa cuando todas las instancias del sub proceso han terminado.

The image shows two configuration panels for a BPMN task. The left panel, titled 'Propiedades Básicas', contains two numeric input fields: 'Cantidad de inicio' (Start quantity) set to 100 and 'Cantidad final' (End quantity) set to 100. The right panel, titled 'Múltiples Instancias' (Multiple Instances), contains three dropdown menus: 'Condición MI' (MI Condition) set to 'Ninguna' (None), 'Orden MI' (MI Order) set to 'Paralelo' (Parallel), and 'Condición de flujo' (Flow condition) set to 'Todo' (All). Both panels have a red rectangular border around their content areas.

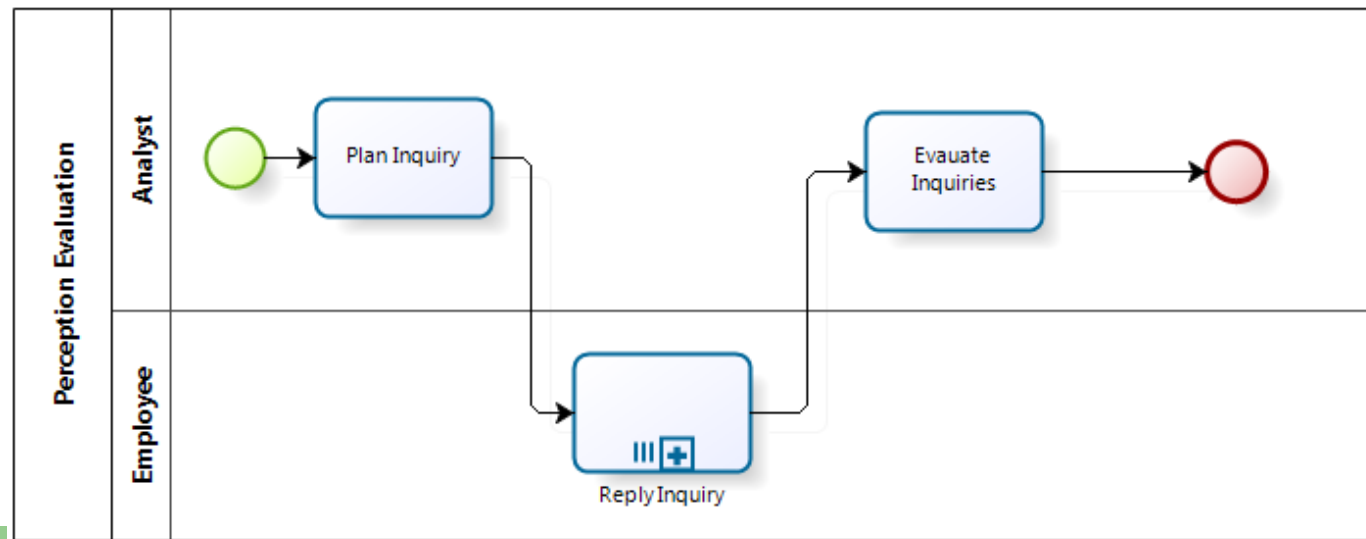
Patrones de múltiples instancias

- Patrón 14 – Múltiples instancias con conocimiento del número de ejecuciones en el tiempo de ejecución
 - Descripción:
 - El patrón permite instanciar una actividad muchas veces durante el proceso. La cantidad de instancias no es conocida hasta el momento de la ejecución del proceso. Las actividades son ejecutadas de forma paralela y es necesario terminar todas las instancias abiertas antes de continuar con la siguiente actividad.
 - Implementación:
 - Para modelar este patrón es necesario utilizar un sub proceso el cual crea las múltiples instancias de una actividad en base a una actividad anterior.



Patrones de múltiples instancias

- Patrón 14 – Múltiples instancias con conocimiento del número de ejecuciones en el tiempo de ejecución
 - Ejemplo:
 - Una compañía decide evaluar la percepción del ambiente de trabajo entre los empleados. Un grupo de empleados es escogido de forma aleatoria para contestar una encuesta. Cuando todas las encuestas han sido respondidas, el analista comienza con la evaluación y análisis de ellas.



Patrones de múltiples instancias

- Patrón 14 – Múltiples instancias con conocimiento del número de ejecuciones en el tiempo de ejecución
 - Para configurar el comportamiento del subproceso sugerencias para la condición que se desea, es necesario definir las siguientes propiedades:
 - **Condición MI** = count(Perception.Employees). Esto significa que se crearan tantas instancias del sub proceso como empleados han sido seleccionados para contestar la encuesta.
 - **Orden MI** = **Paralelo**. Las instancias del sub proceso se ejecutan en paralelo.
 - **Condición de Flujo** = **Todo**. El proceso terminara cuando todas las instancias hayan terminado.

Propiedades de elemento

Básicos Extendidos **Avanzados** Acciones de Presentación

Propiedades Básicas

Cantidad de inicio 1

Cantidad final 1

Múltiples Instancias

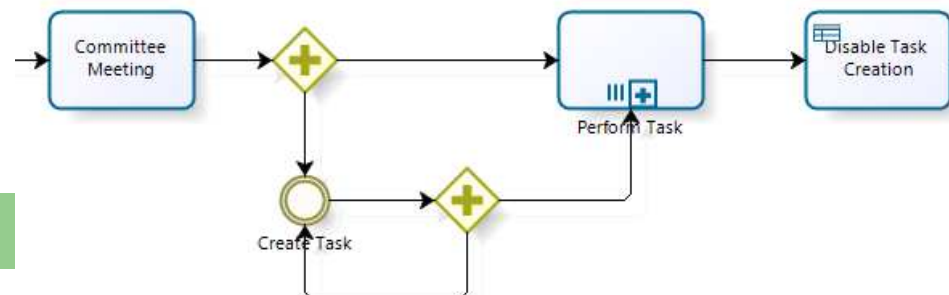
Condición MI count(perceptionempl...

Orden MI Paralelo

Condición de flujo Todo

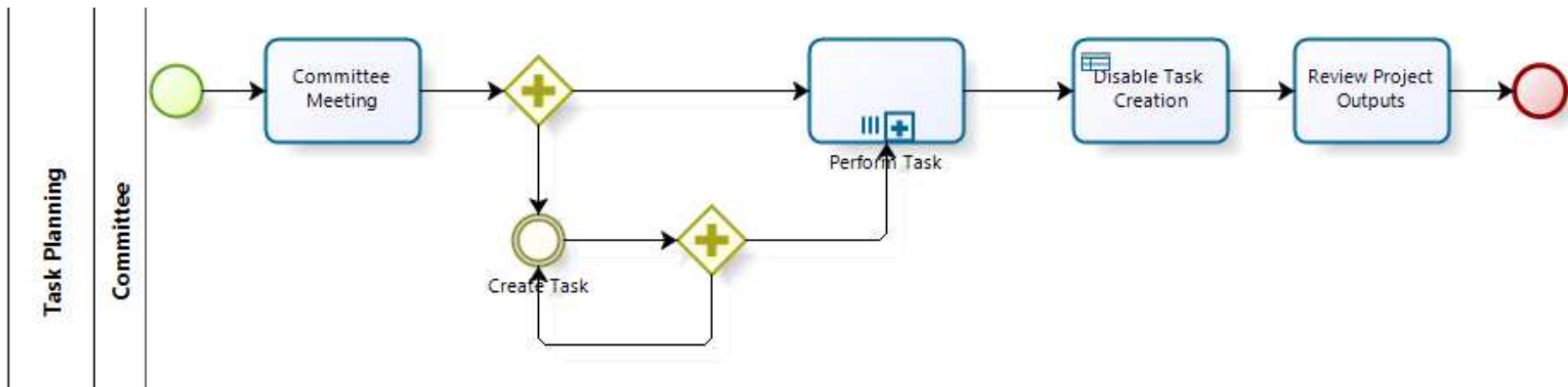
Patrones de múltiples instancias

- Patrón 15 – Múltiples instancias sin conocimiento del número de ejecuciones
 - Descripción:
 - Este patrón permite a una actividad ser instanciada muchas veces durante el proceso. La cantidad de instancias no es conocida ni durante la etapa de diseño ni en el tiempo de ejecución. Las actividades son ejecutadas al mismo tiempo, pero mientras se están ejecutando, se pueden crear nuevas instancias de ellas. Es necesario terminar todas las instancias para poder continuar con el flujo del proceso.
 - Implementación:
 - En una actividad anterior se crean un número inicial de las instancias (actividades), luego, un evento es utilizado para permitir la creación de nuevas instancias. Una vez las tareas han sido completadas, el evento es deshabilitado y no se permite crear nuevas instancias.



Patrones de múltiples instancias

- Patrón 15 – Múltiples instancias sin conocimiento del número de ejecuciones
 - Ejemplo:
 - El equipo del proyecto realiza una reunión de comité para empezar el proyecto. Durante la reunión se planean varias tareas que se deben realizar posteriormente. Sin embargo, una situación inesperada puede surgir y nuevas tareas deben ser realizadas. El equipo del proyecto puede crear nuevas tareas, de ser requerido, antes de que todas las instancias hayan terminado.



Patrones de múltiples instancias

- Patrón 15 – Múltiples instancias sin conocimiento del número de ejecuciones
 - Para configurar el comportamiento del subproceso sugerencias para la condición que se desea, es necesario definir las siguientes propiedades:
 - **Condición MI = count(Project.Task)**. Esto significa que se crearan tantas instancias del sub proceso como tareas han sido creadas.
 - **Orden MI = Paralelo**. Las instancias del sub proceso se ejecutan en paralelo.
 - **Condición de Flujo = Todo**. El proceso terminara cuando todas las instancias hayan terminado.

Propiedades de elemento

Básicos Extendidos **Avanzados** Acciones de Presentación

Propiedades Básicas

Cantidad de inicio 1

Cantidad final 1

Múltiples Instancias

Condición MI count(project.task)

Orden MI Paralelo

Condición de flujo Todo

Patrones Basados en Estados

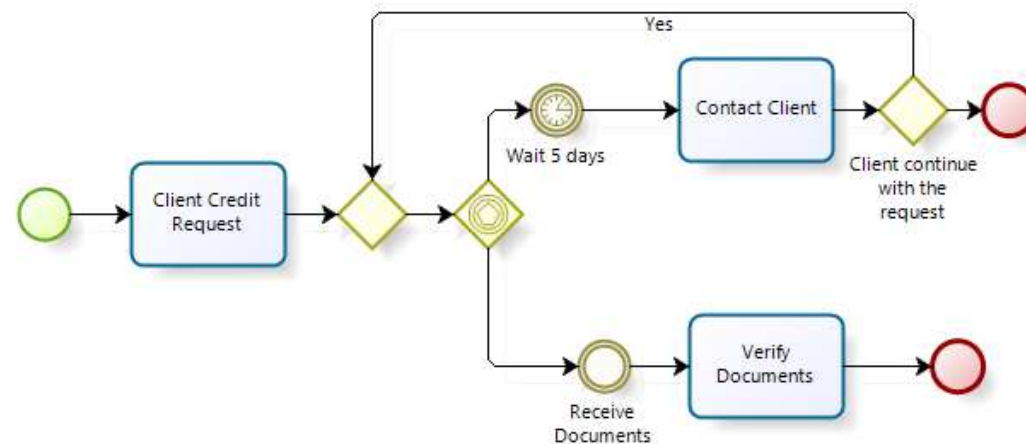
- Patrón 16 – Decisión Diferida

- Descripción:

- El patrón describe un punto en el proceso donde una rama es escogida de varias disponibles. La decisión no es tomada hasta que comience el procesamiento de uno de los flujos alternativos. Cuando una rama es escogida, las demás se deben deshabilitar.

- Implementación:

- Para implementar el patrón es necesario utilizar una **compuerta exclusiva basada en eventos**. Esta compuerta representa un punto en el proceso donde solo una de las ramas disponibles debe ejecutar. Las ramas restantes deben ser deshabilitadas.

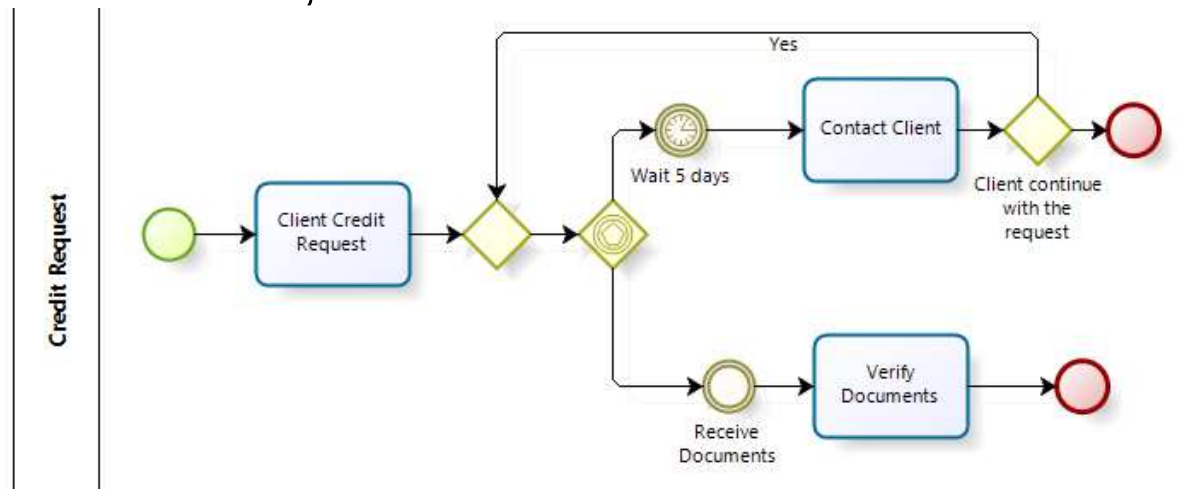


Patrones Basados en Estados

- Patrón 16 – Decisión Diferida

- Ejemplo:

- Cuando un cliente realiza una solicitud de crédito es necesario solicitarle diferentes documentos; Si el cliente no trae los documentos antes de 5 días se le debe contactar y preguntar si continúa o no en el proceso. Si el cliente no continúa en el proceso no es necesario esperar por los documentos y el proceso debe terminar. Si el cliente trae los documentos, no es necesario contactarlo.



Patrones Basados en Estados

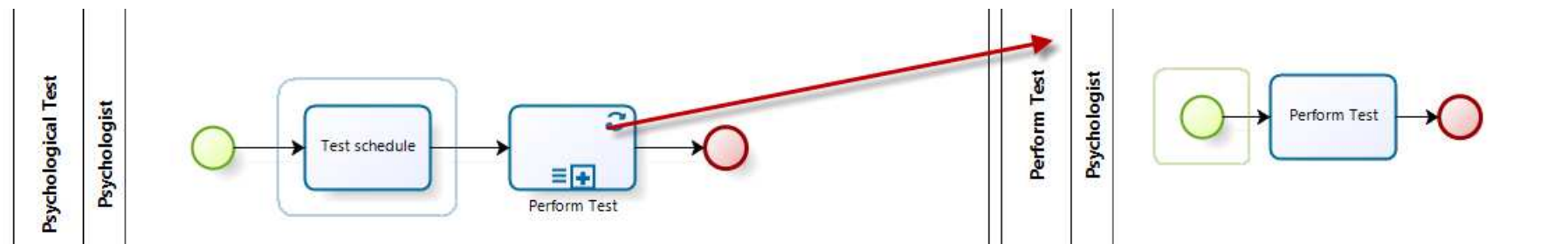
- Patrón 17 – Ejecución Paralela Intercalada

- Descripción:

- Un conjunto de actividades tienen un orden de ejecución definido. Cada actividad puede ser ejecutada una vez y se puede completar en cualquier momento de acuerdo al orden definido. Sin embargo, como requisito adicional, dos actividades no pueden ser ejecutadas al mismo tiempo

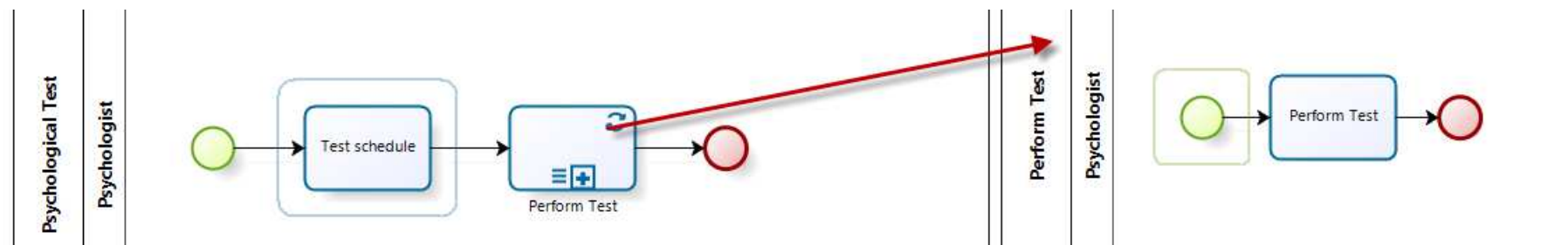
- Implementación:

- Para modelar el patrón se utiliza un sub proceso múltiple, el sub proceso solo incluye una actividad.



Patrones Basados en Estados

- Patrón 17 – Ejecución Paralela Intercalada
 - Ejemplo:
 - El Jefe de Recursos Humanos estable que durante los procesos de selección los aspirantes deben presentar tres pruebas de aptitud: una prueba psicológica, un test de inteligencia y un test de conocimientos específicos.
 - El orden en que se realizan las pruebas es importante. El test de inteligencia debe ser realizado antes de la prueba psicológica. La prueba de conocimientos debe ser realizada en cualquier momento. No es posible realizar dos pruebas al mismo tiempo. La secuencia en que las pruebas se realizan es definida en la primera actividad del proceso



Patrones Basados en Estados

- Patrón 17 – Ejecución Paralela Intercalada
 - Para configurar el comportamiento del subprocesso realizar pruebas para la condición que se desea, es necesario definir las siguientes propiedades:
 - **Cantidad de Inicio** = 3. Esto significa que el proceso debe ser instanciado 3 veces.
 - **Cantidad Final** = 3. El sub proceso se considera como terminado cuando se hayan completado todas (en este caso) las 3 instancias.
 - **Condición MI** = **Ninguna**. Esto quiere decir que no hay condición para el número de instancias para el sub proceso.
 - **Orden MI** = **Secuencial**. Las instancias del sub proceso se ejecutan en forma secuencial.

The image shows two side-by-side configuration panels from a BPMN modeling tool. The left panel, titled 'Propiedades Básicas', contains two input fields: 'Cantidad de inicio' with a value of 3 and 'Cantidad final' also with a value of 3. The right panel, titled 'Múltiples Instancias', contains two settings: 'Condición MI' with a dropdown menu set to 'Ninguna' and 'Orden MI' with a dropdown menu set to 'Secuencial'.

Patrones Basados en Estados

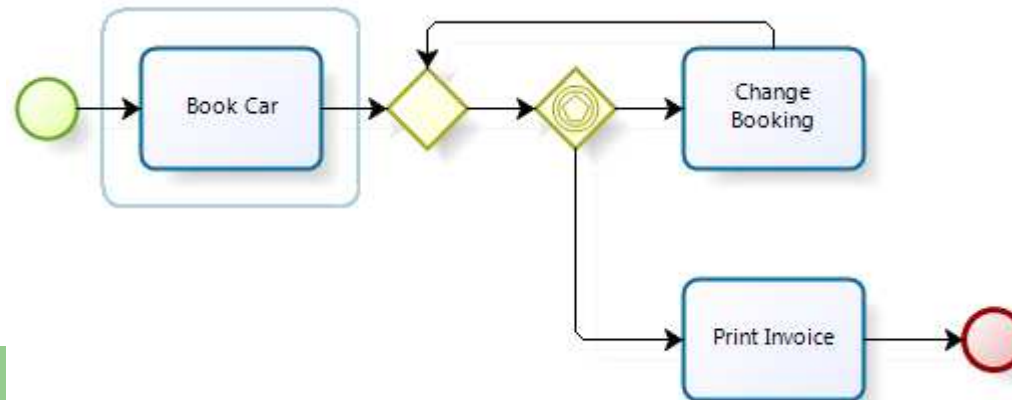
- Patrón 18 – Milestone

- Descripción:

- El patrón define que una actividad es habilitada solamente cuando el proceso alcance un estado específico (Hito). Si el proceso ha avanzado más allá de ese estado, la tarea no puede volver a ser habilitada.

- Implementación:

- El patrón utiliza una compuerta exclusiva basada en eventos para controlar la disponibilidad de la actividad.

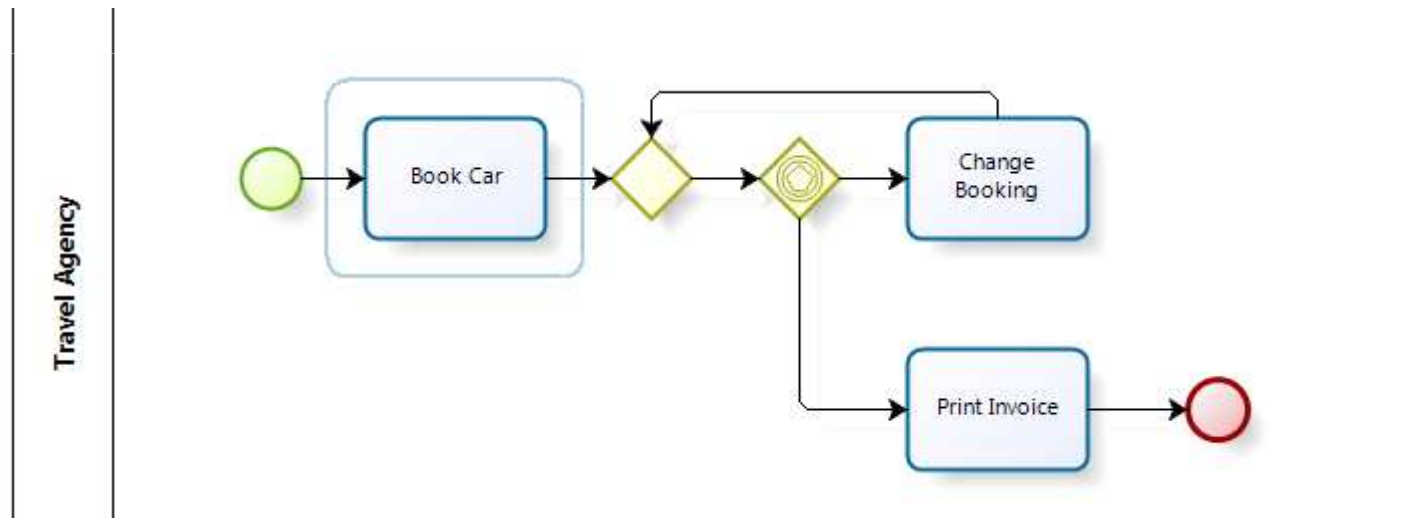


Patrones Basados en Estados

- Patrón 18 – Milestone

- Ejemplo:

- En una agencia de viajes es posible cambiar las fechas de las reservas de vuelos, autos y hoteles mientras las facturas no hayan sido impresas.



Patrones de Cancelación

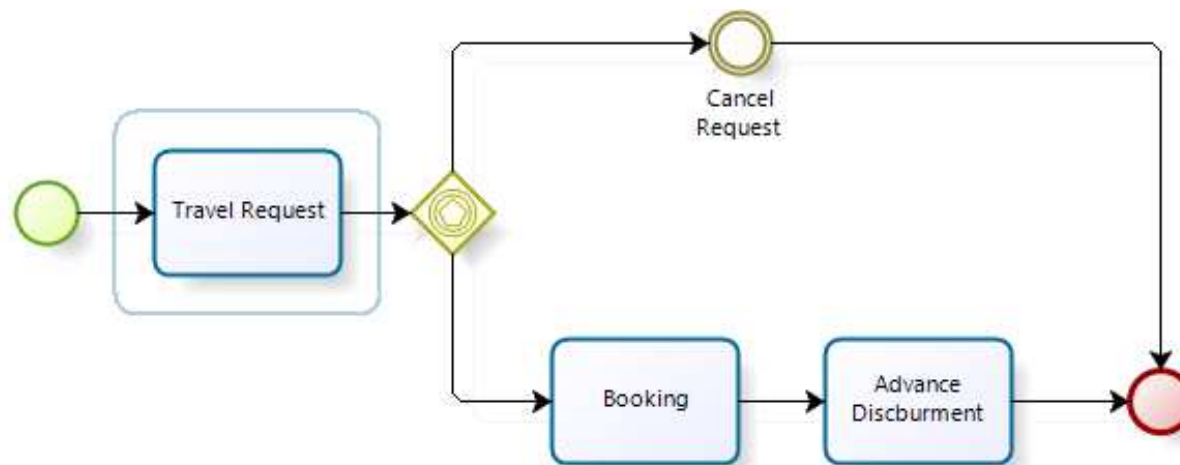
- Patrón 19 – Cancelación de Tareas

- Descripción:

- El patrón de Cancelación de Tareas da la posibilidad de cancelar una tarea que está siendo ejecutada.

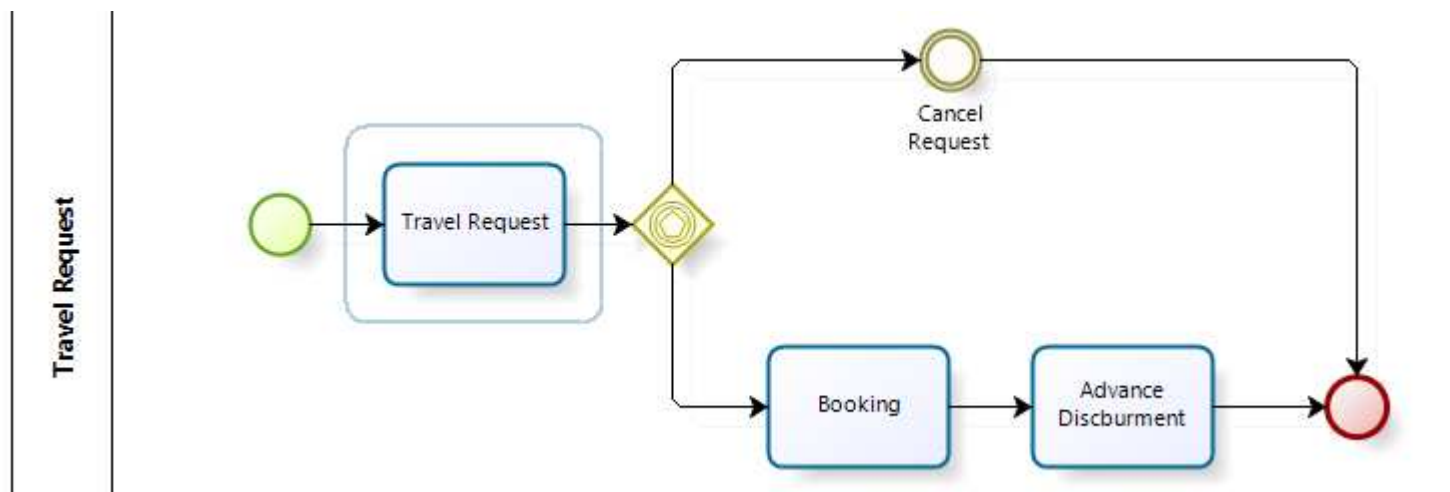
- Implementación:

- El patrón es modelado con una **compuerta basada en eventos y un evento intermedio**. La compuerta habilita una actividad y el evento que cancela el proceso. Solo una de estas dos cosas puede pasar.



Patrones de Cancelación

- Patrón 19 – Cancelación de Tareas
 - Ejemplo:
 - Durante una solicitud de viajes es necesario reservar varias cosas, por ejemplo, hotel, auto, vuelo y entregar un adelanto al empleado. El empleado debe poder cancelar en viaje en cualquier momento de ser necesario. Si se cancela el viaje no se debe realizar reservas, y solo es posible cancelar el viaje antes de entregar el adelanto al empleado



Patrones de Cancelación

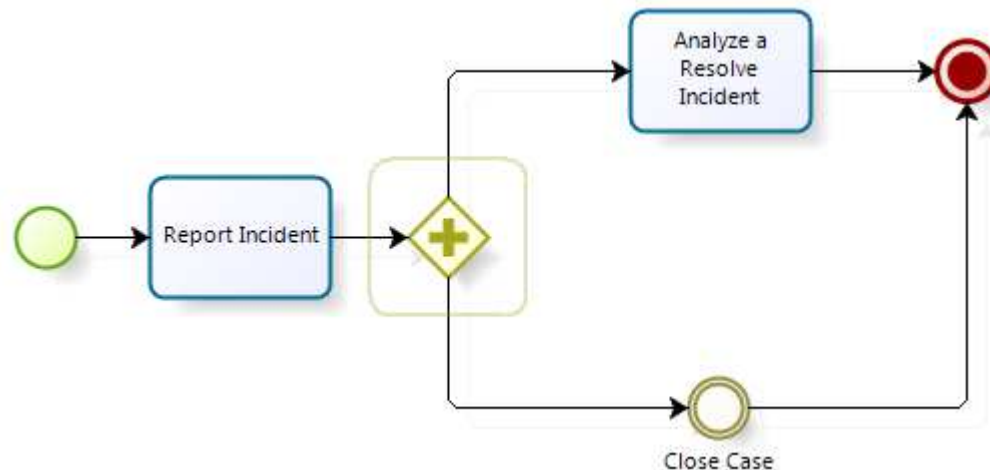
- Patrón 20 – Cancelación del Caso

- Descripción:

- El patrón describe la eliminación de una instancia completa del proceso. Esto incluye tareas que están siendo ejecutadas y las tareas que todavía no lo han sido.

- Implementación:

- Para implementar el patrón se utiliza un **evento de fin terminal**. Una vez que el proceso alcance el fin terminal, las actividades pendientes son canceladas y el caso termina.

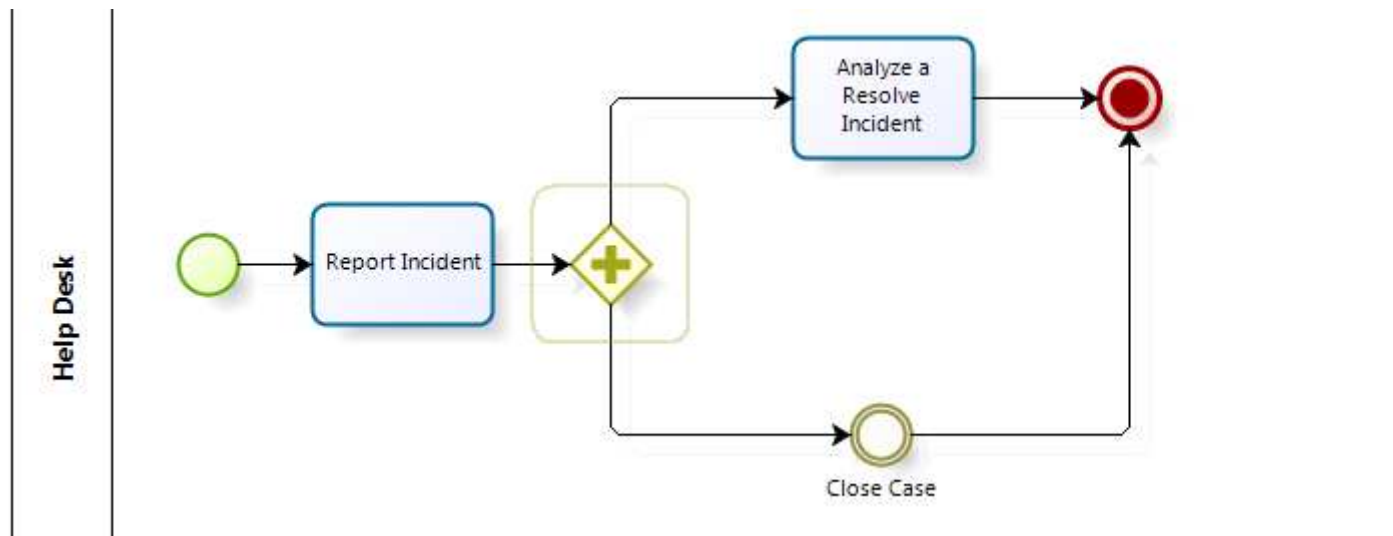


Patrones de Cancelación

- Patrón 20 – Cancelación del Caso

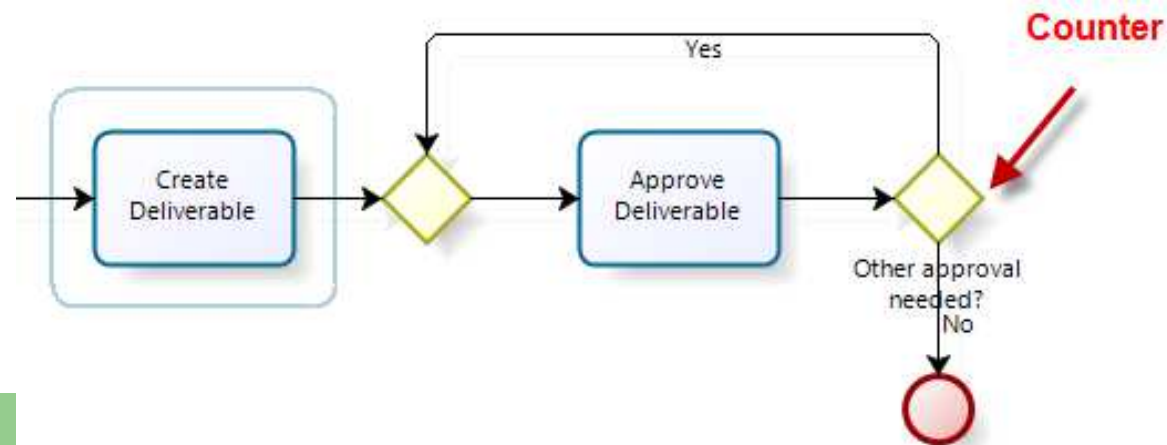
- Ejemplo:

- Cuando un incidente ocurre, este debe ser reportado a la Mesa de Ayuda de la compañía. Luego de reportar el incidente, la Mesa de Ayuda debe resolverlo y tener la posibilidad de cancelar el caso en cualquier momento.



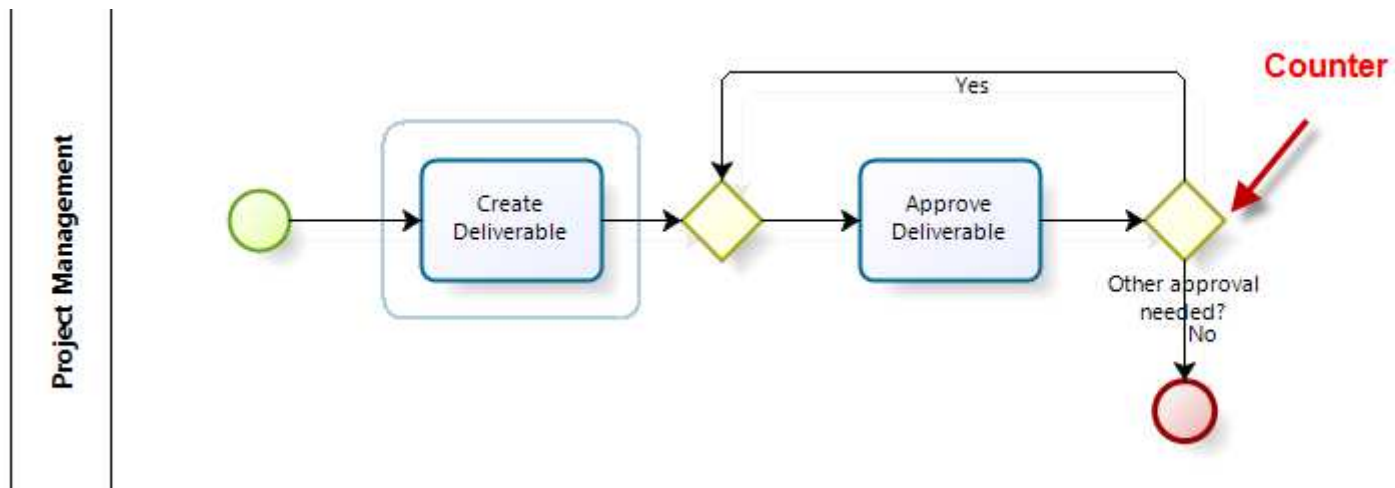
Patrones de Iteraciones

- Patrón 10 – Ciclos Arbitrarios
 - Descripción:
 - El patrón modela un punto en el flujo del proceso donde una o más actividades son realizadas varias veces.
 - Implementación:
 - El patrón utiliza una compuerta exclusiva que controla el número de veces que se ha aprobado el entregable, cuando se cumpla la condición (número de aprobaciones requeridas) el proceso continúa por el flujo normal.



Patrones de Iteraciones

- Patrón 10 – Ciclos Arbitrarios
 - Ejemplo:
 - Los entregables de un proyecto necesitan la aprobación de varias personas. El número de aprobaciones es definido por el Gerente del Proyecto



Patrones de Terminación

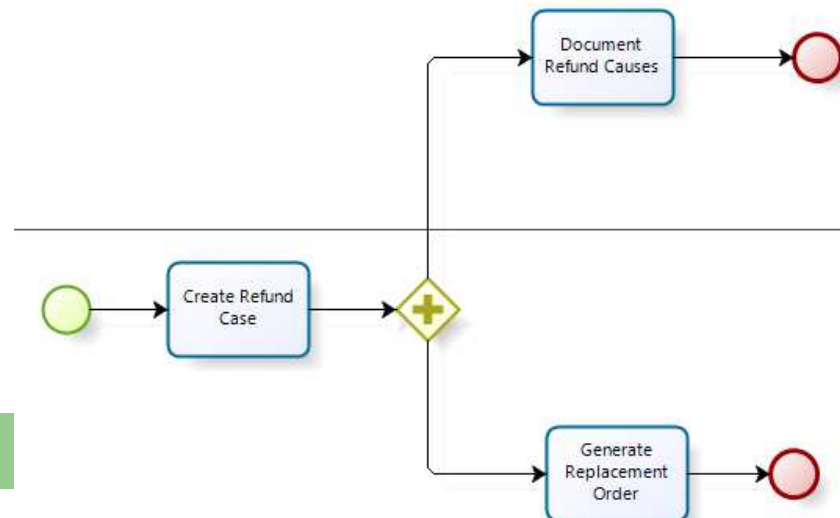
- Patrón 11 – Terminación Implícita

- Descripción:

- El patrón es utilizado para determinar cuándo la instancia de un proceso se considera completa. Un determinado proceso (o sub proceso) debe terminar cuando no hay tareas pendientes o cuando no hay posibilidad de crear nuevos trabajos en el futuro.

- Implementación:

- El patrón utiliza evento al final de los caminos que deben ser completados. El proceso se considera terminado cuando todos los eventos de fin son alcanzados

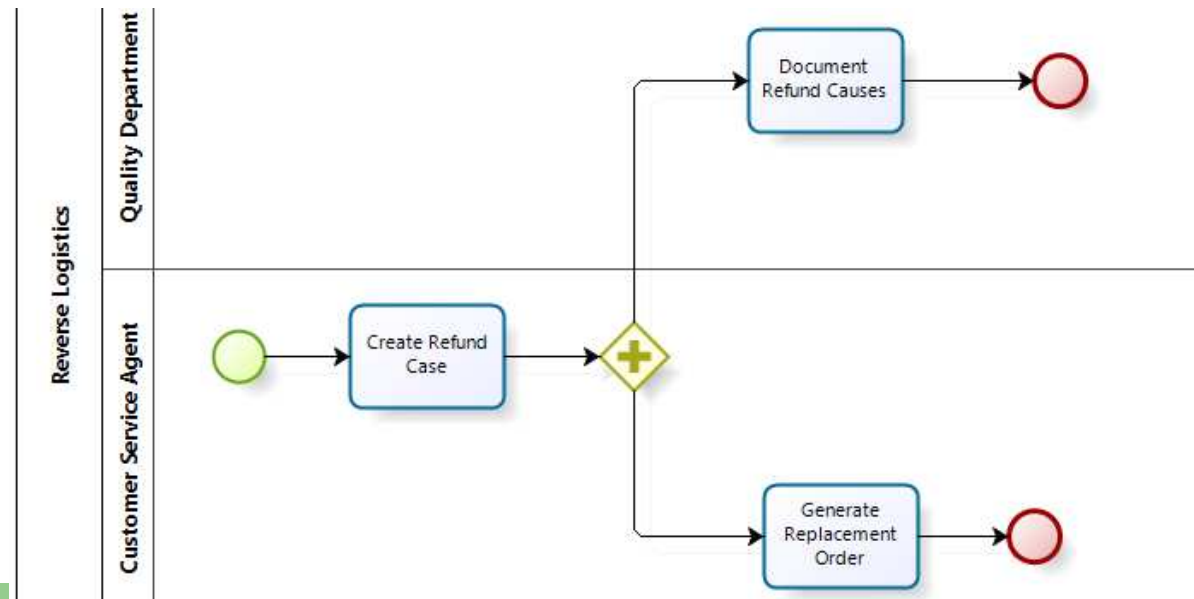


Patrones de Terminación

- Patrón 11 – Terminación Implícita

- Ejemplo:

- Considere la Logística Inversa en una empresa manufacturera. Cuando se recibe una solicitud de cambio se deben realizar dos actividades para que el proceso pueda terminar. La primera actividad es generar una orden de cambio y la segunda es la de documentar las causas de la devolución.



Patrones de Terminación

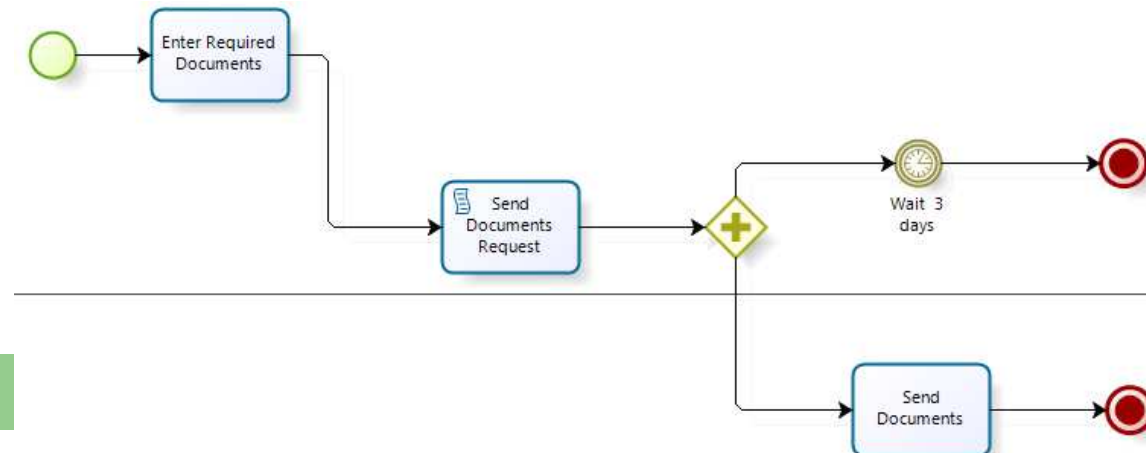
- Patrón 43 – Terminación Explícita

- Descripción:

- La instancia de un proceso o de un sub proceso debe terminar si alcanza cierto estado. Generalmente se especifica utilizando un nodo de final específico. Cuando se alcanza el nodo, las tareas o trabajo faltante son cancelados y la instancia del proceso es completada de forma exitosa.

- Implementación:

- El patrón utiliza eventos de Fin Terminal al final de cada una de las ramas habilitadas. El proceso se considera como terminado cuando alguno de los eventos es alcanzado.



Patrones de Terminación

- Patrón 43 – Terminación Explícita

- Ejemplo:

- Suponga un proceso de Solicitud de Crédito. El proceso incluye un sub proceso llamado Solicitar Documentos donde un analista de crédito determina qué papeles deben ser enviados por el cliente para otorgar el crédito, luego, un correo electrónico es enviado al cliente solicitando los documentos. Si luego de 3 días el cliente no ha entregado los documentos, la solicitud se cancela y el sub proceso debe terminar.

